Lecture 09: Database Implementation

Data Dictionary and DDL

ITCS216 Fundamentals of Database Management Systems

Semester 1/2026

Asst. Prof. Dr. Preecha Tangworakitthaworn preecha.tan@mahidol.ac.th
Asst. Prof. Dr. Siripen Pongpaichet siripen.pon@mahidol.ac.th
Asst. Prof. Dr. Jidapa Kraisangka jidapa.kra@mahidol.ac.th

Lesson Learning Outcome

After this lesson, students are able to:

- Create a data dictionary that accurately represents a relational schema, including attributes, data types, keys, constraints, domains, and references.
- Translate a relational schema into database tables using SQL Data Definition Language (DDL).
- define and apply primary keys, composite keys, foreign keys, and other integrity constraints.
- Modify and remove database objects using ALTER and DROP.
- Insert, update, and delete table records using SQL Data Manipulation Language (DML).
- Use MySQL Server and MySQL Workbench to create, execute, and save SQL scripts.

Database Topics

Knowing Database and How to Develop Data Model for Business Application PART I

- Fundamental Concepts of DBMS
- The Three-Schema Architecture
- Entity-Relationship Model
- Developing an E-R Diagram
- Relational Data Model and Relational Algebra
- Developing a Relational Database Schema from an E-R Diagram
- Key & Integrity Constraints Basic Concepts

More Inside Database Design, Implementation, and Management PART II

- **SQL Programming** ITCS393 Database Lab
- Transaction Management & Concurrency Control

ITCS413: Database Design

ITCS457 DSS

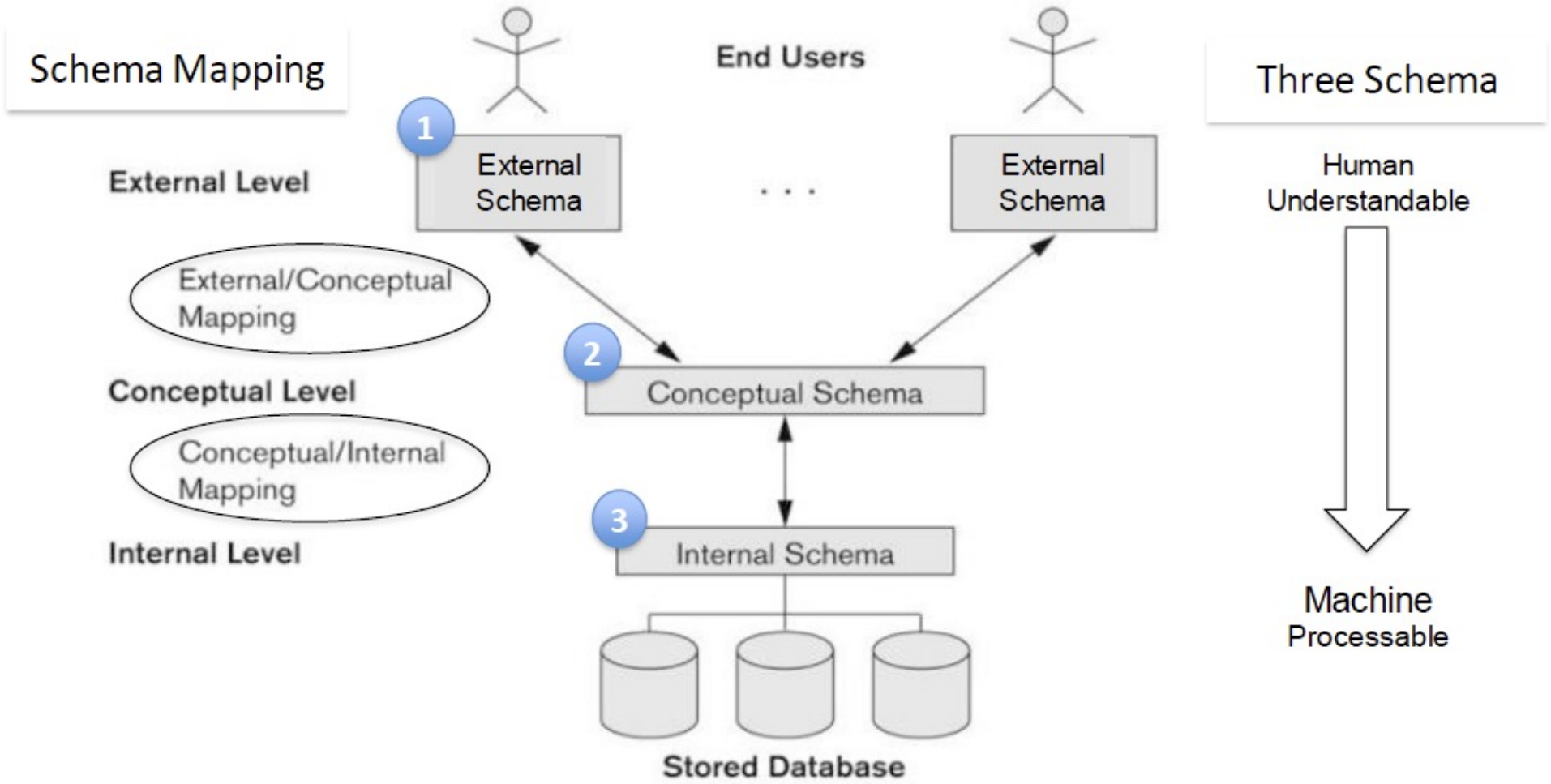
- Normalization Theory for Relational Database Design
- Database Design Process
- Managing Database and SQL Performance.
- Databases for Decision Support.



Database Concept

Recommend

Three Schema Architecture



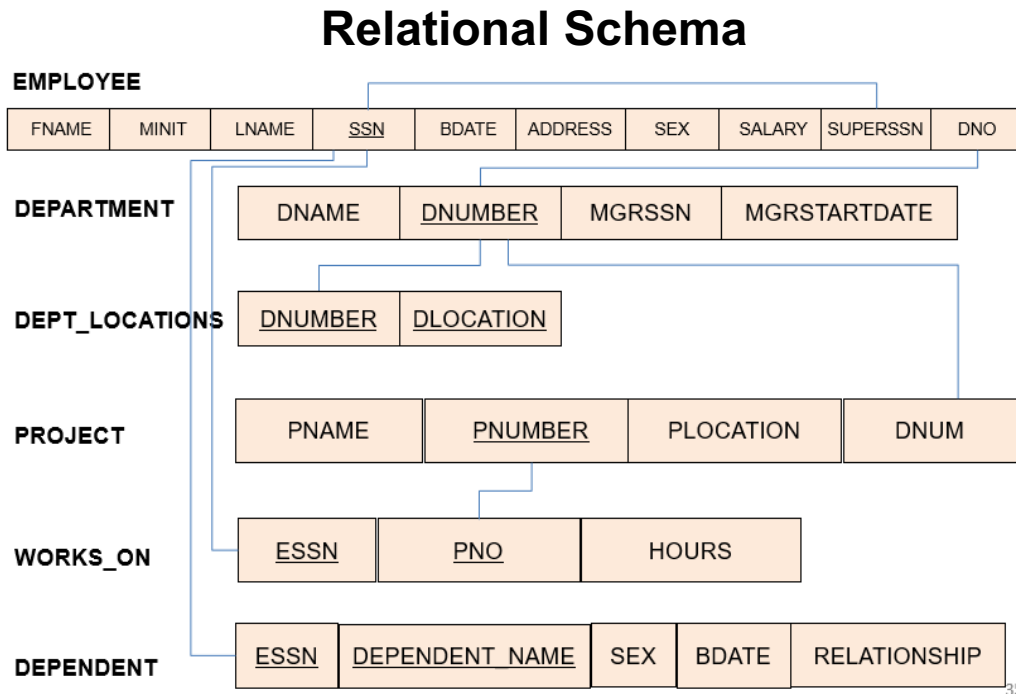
Mahidol University

Database Implementation



Relational Data Model (Schema)

- The representation of DB as seen by the DBMS
- Match the conceptual models' characteristics and constraints to the implementation model



+ Key and Constraints

Relational Data Model (Schema)

- The **schema** of a table is the table name, and its attributes

```
Department (dnumber, dname, location)
```

- A primary **key** (PK) is an attribute whose values are unique and cannot be NULL.
  We usually underline a key in the schema.
  (more detail later)

```
Department (dnumber, dname, location)
```

Table in Relational Database

A **tuple** (or **row** or **record**) is a single entry in the table having the attributes specified by the schema

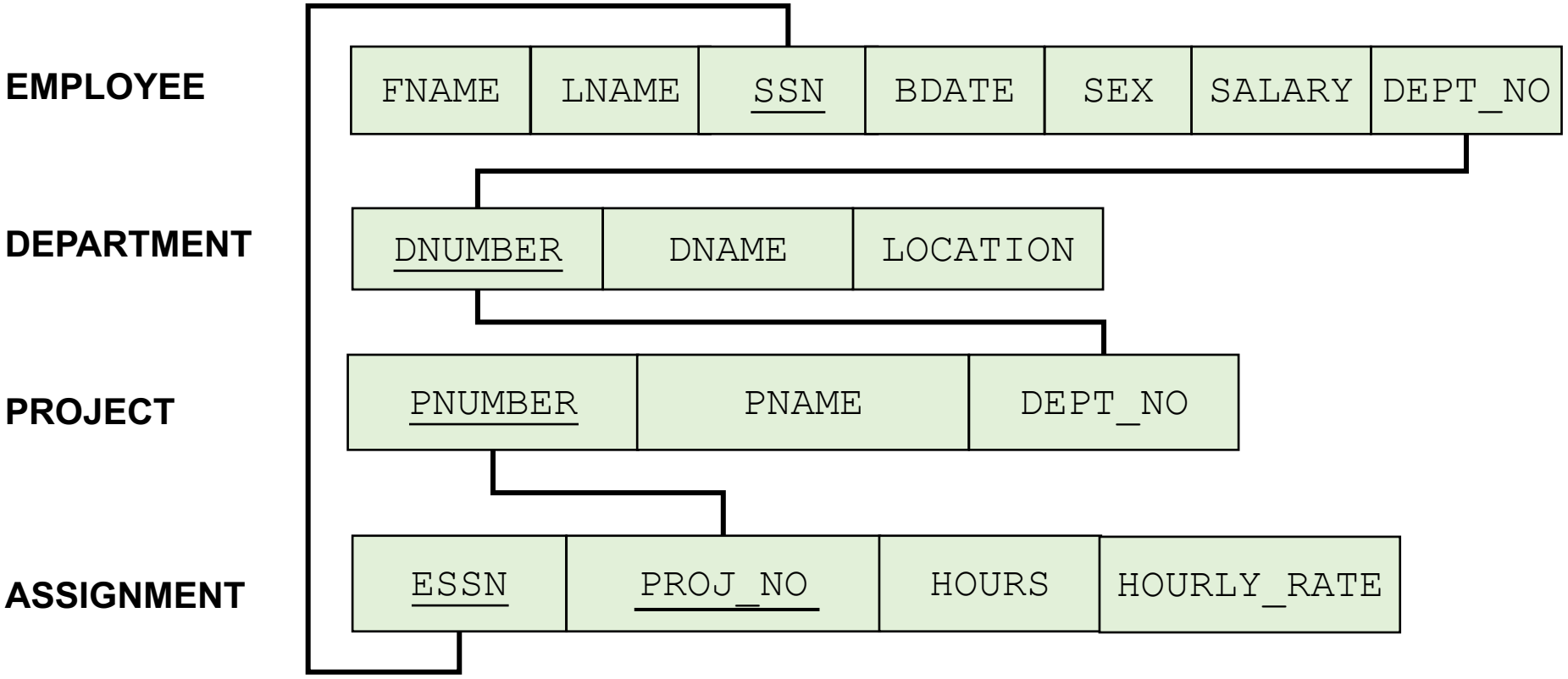
DNUMBER	DNAME	LOCATION
1	Administration	Houston
4	Headquarters	Stafford
5	Research	Irvine
6	Research	NULL

A **relation** or **table** is a set of tuples

An **attribute** (or **column**) is a typed data entry presented in each tuple in the relation

EX: tinycompany Relational Schema

(Simplify Version)



Data Dictionary

- A **data dictionary** is a detailed catalog describing all data elements within a database, for examples,
  - What the attributes means (description)
  - How the attributes are stored (data types)
  - How the attributes relate to one another, i.e., foreign keys

- Serves as a **blueprint** for implementation and helps **developers, DBAs, and analysts** understand data definitions and constraints.

Components in Data Dictionary

Element	Description
Table Name	A relation name (usually from your relational data model)
Attribute Name	The attribute name belonged to the table
Contents / Description	Explanation of the attributes
Data Type	Defines the type of data stored (e.g., `INT`, `VARCHAR`, `DATE`)
Format	Specifies the pattern or structure for data input/output of the given attributes (e.g., `yyyy-mm-dd`).
Nullable	Indicates whether the attribute can contain `NULL` (empty) values.
Range / Domain	Acceptable range or set of valid values for the attribute.
Key	Identifies the primary key (PK) or foreign key (FK) used for data relationships.
Referenced Table	Indicates the table and column linked through a foreign key.
Default Value	Predefined value assigned if no data is entered.

Note: These components are commonly included in a data dictionary, but are not limited to those listed here.

Example: Data Dictionary for `tinycompany`

PK = Primary Key FK = Foreign Key

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Referenced Table
Department	dnumber	Department's number	int	x		1 to 20	PK	
	dname	Department's name	varchar(20)	Xxxxxx				
	location	Department's main location	varchar(100)	Xxxxxx	Y			
Employee	fname	Employee's first name	varchar(20)	Xxxxxx				
	lname	Employee's last name	varchar(20)	Xxxxxx				
	ssn	Social security number	char(9)	xxxxxxxxx			PK	
	bdate	Employee's birthday	date	yyyy-mm-dd				
	sex	Employee's gender	char(1)	X		M,F		
	salary	Salary	decimal(12,2)	1234567890.00	Y			
	dept_no	Department's number	int	x	Y		FK	dnumber [Department]
Project	pnumber	Project's number	int	x			PK	
	pname	Project's name	varchar(50)	Xxxxxx				
	dept_no	Department's number	int	x			FK	dnumber [Department]
Assignment	essn	Empolyee's SSN	char(9)	xxxxxxxxx			PK, FK	ssn [Employee]
	proj_no	Project's number	int	x			PK, FK	pnumber [Project]
	hours	Number of hours spent	decimal(9,2)	1234567.89	Y			
	hourly_rate	Hourly rate	decimal(9,2)	1234567.89	Y			

Commonly Used Data Types in RDBMS

Data Type	Explanation	Example Usage
CHAR(n)	*Fixed-length* character string. Always stores exactly `n` characters, padding with spaces if shorter.	`CHAR(5) → 'ICT  ', CHAR(7) → '6888742'`
NCHAR(n)	*Fixed-length Unicode* character string for multilingual text (supports languages like Thai, Chinese, etc.).	`NCHAR(10) → 'มหิดล    '`
VARCHAR(n)	*Variable-length* character string. Uses only as much space as needed (up to n).	`VARCHAR(50) → 'Database Systems'`
NVARCHAR(n)	*Variable-length Unicode* string for multilingual text, supports Unicode characters.	`NVARCHAR(100) → 'เทคโนโลยีสารสนเทศ'`
INT	Integer (whole number) type for numeric data without decimals.	`INT → 12345`
DECIMAL(p, s)	Fixed-point number with `p` total digits and `s` digits after the decimal point.	`DECIMAL(10,2) → 12345.67`
DATE	Stores calendar date only (year, month, day).	`DATE → '2025-10-07'`
DATETIME	Stores both date and time (year, month, day, hour, minute, second).	`DATETIME → '2025-10-07 10:30:00'`
BIT	Boolean-like field storing 1 (true), 0 (false), or NULL.	`BIT → 1 (true)`

Build a Database using SQL

- Use your relational model and data dictionary to create a working database using SQL commands.
- Require a database management software (DBMS), for examples,
  - MySQL / MariaDB for lightweight, web-focused applications.
  - PostgreSQL for research, analytics, advanced constraints.
  - SQL Server or Oracle for enterprise-level operations.



Mahidol University

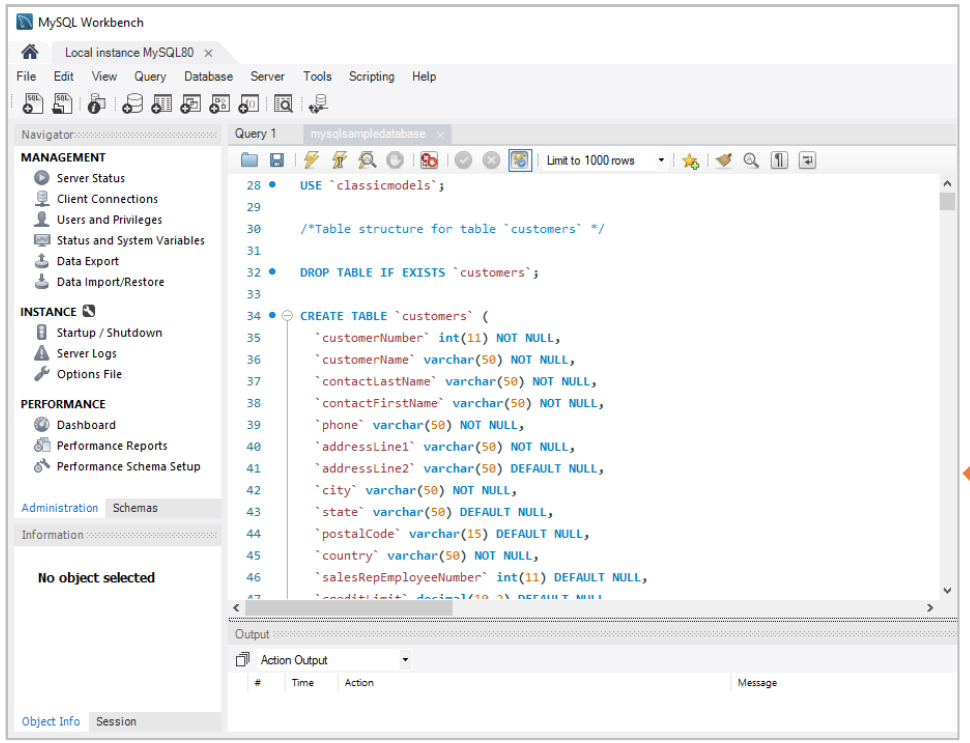


Introduction to SQL

Basic Concepts

Structural Query Language

- "S-Q-L", often pronounced as "sequel"
- SQL is a standard language for querying and manipulating data in database

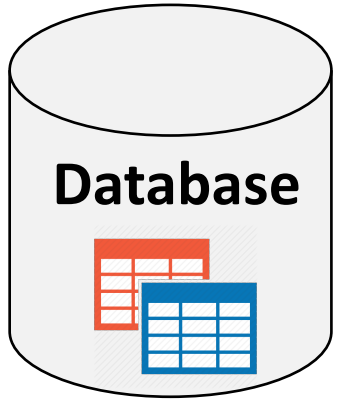




Manipulate DB and Schema

Insert / Update/ Delete Data

Retrieve Data



PostgreSQL MariaDB MySQL Microsoft SQL Server ORACLE DATABASE

Structured Query Language (SQL)

There are several categories of SQL

- Data Definition Language (DDL)
  - Define and manipulate database, relational schema, and other database objects
  - Example: `CREATE, ALTER, DROP`
- Data Manipulation Language (DML)
  - Manage data in tables
  - Example: `INSERT, UPDATE, DELETE`
- Data Query Language (DQL)
  - Perform queries on data in the schema
  - Retrieve data from the database
  - `SELECT`

Mahidol University



SQL: Data Definition Language

DDL: Data Definition Language

- Used for defining database objects, such as, databases, tables, indexes, views, stored procedures.
  - `CREATE` - Used to create a new database object.
  - `ALTER` - Used to modify an existing database object.
  - `DROP` - Used to delete an entire database object.

Working with Databases

- **Create** a database with a specified name

```
CREATE DATABASE [IF NOT EXISTS] databasename;
```

Example:

```
CREATE DATABASE IF NOT EXISTS tinycompany;
```



Working with Databases

- Show all available databases

```
SHOW DATABASES; -- Do not forget s at the end
```

- Use or set the current working database

```
USE databasename; -- e.g. USE tinycompany;
```

- Drop (delete) a database permanently.

```
DROP DATABASE [IF EXISTS] databasename;
```

Example:

```
SHOW DATABASES;
USE tinycompany;
DROP DATABASE IF EXISTS tinycompany;
```

Working with Tables

- Create a table with primary keys
- Create a table with foreign keys
- Modify an existing table
- Drop a table

Create a Table / Schema

- Create a table from your designed relational schema
  - Use one line per column (attribute) definition
  - Use spaces or tabs to line up attribute characteristics and constraints

```
CREATE TABLE [IF NOT EXISTS] tablename (
attrname1   datatype(size) [constraintname],
attrname2   datatype(size) [constraintname],
...

attrnameN   datatype(size) [constraintname],
[PRIMARY KEY  (attrPK1 [, attrPK2]) ]
REFERENCES tablename]  [, CONSTRAINT
constraintname ]
);
```

Create a Table/Schema - Constraints

- SQL constraints are used to specify rules for the data in a table
- Used when `CREATE` or `ALTER` (modify) table.
  - `NOT NULL` - a column cannot store `NULL` value
  - `UNIQUE` - each row for a column must have a unique value
  - `PRIMARY KEY` – a column or more columns have an unique identity and cannot be `NULL` value (`NOT NULL` and `UNIQUE` constraints)

Example: CREATE `department` Table

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Referenced Table
Department	dnumber	Department's number	int	x		1 to 20	PK	
	dname	Department's name	varchar(20)	Xxxxxx				
	location	Department's main location	varchar(100)	Xxxxxx	Y			

DEPARTMENT

DNUMBER	DNAME	LOCATION

Option 1: Declare a primary key constraint immediately after the attribute definition.
Common for **single-column primary keys**.
Noted that NOT NULL and UNIQUE can be declared.

```
CREATE TABLE department(
      dnumber      INT                NOT NULL UNIQUE PRIMARY KEY,
      dname        VARCHAR(20)        NOT NULL,
      location     VARCHAR(100),
);
```

Example: CREATE `department` Table

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Referenced Table
Department	dnumber	Department's number	int	x		1 to 20	PK	
	dname	Department's name	varchar(20)	Xxxxxx			PK	
	location	Department's main location	varchar(100)	Xxxxxx	Y			

DEPARTMENT

DNUMBER	DNAME	LOCATION

Option 2: Declare a primary key at the end of the attribute list. Require for defining a composite key.
Noted that NOT NULL can be declared, but UNIQUE cannot apply for Composite PKs.

```
CREATE TABLE department(
      dnumber              INT                 NOT NULL,
      dname                VARCHAR(20)         NOT NULL,
      location             VARCHAR(100),
      PRIMARY KEY          (dnumber, dname)
);
```

Example: CREATE `department` Table

Table Name	Attribute Name	Contents	Type	Format	Nullable	Range	Key	FK Referenced Table
Department	dnumber	Department's number	int	x		1 to 20	PK	
	dname	Department's name	varchar(20)	Xxxxxx			PK	
	location	Department's main location	varchar(100)	Xxxxxx	Y			

DEPARTMENT

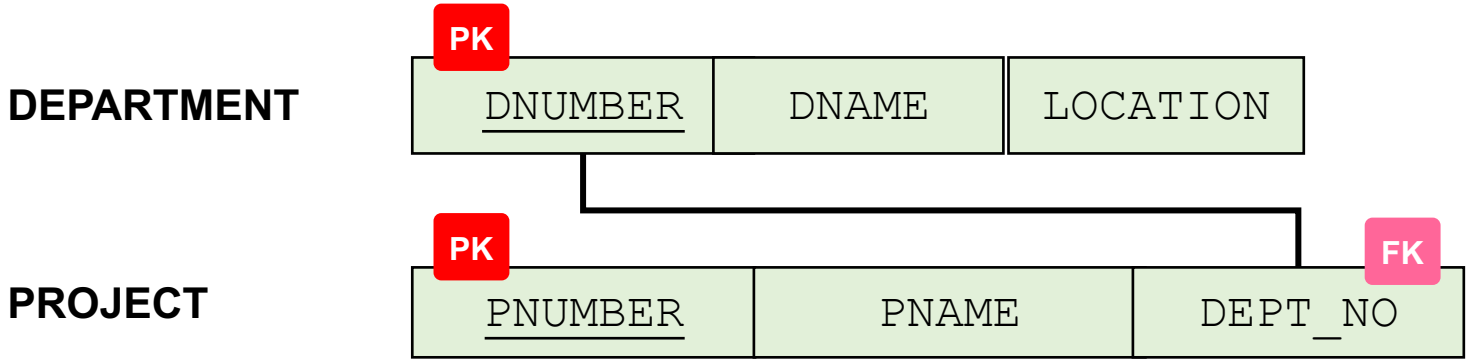
DNUMBER	DNAME	LOCATION

Option 3: Declare a primary key Constraint at the end of the attribute list. Require for defining a composite key.
Noted that NOT NULL can be declared, but UNIQUE cannot apply for Composite PKs.

```
CREATE TABLE department(
      dnumber             INT                 NOT NULL,
      dname               VARCHAR(20)         NOT NULL,
      location            VARCHAR(100),
      CONSTRAINT PK_Dept PRIMARY KEY (dnumber, dname)
);
```

Constraint name

Create a Table/Schema with Foreign Keys



PROJECT

PNUMBER	PNAME	DEPT_NO
1	ProductX	5
2	ProductY	5
10	Computerization	4
20	Recognization	1

DEPARTMENT

DNUMBER	DNAME	LOCATION
1	Administration	Houston
4	Headquarters	Stafford
5	Research	Irvine
6	Research	NULL

PK Entity Integrity

FK Referential Integrity

Example: Create a `project` Table

Project	pnumber	Project's number	int	x			PK	
	pname	Project's name	varchar(50)	Xxxxxx				
	dept_no	Department's number	int	x			FK	dnumber [Department]

PROJECT

PK		FK
PNUMBER	PNAME	DEPT_NO

```
CREATE TABLE project(
      pnumber      INT           ,
      pname        VARCHAR(50) NOT NULL,
      dept_no      INT,
      CONSTRAINT PK_Proj PRIMARY KEY (pnumber),
      CONSTRAINT FK_DeptProj FOREIGN KEY (dept_no)
      REFERENCES department(dnumber)
);
```

Constraint name



Can we create a `project` table before a `department` table?

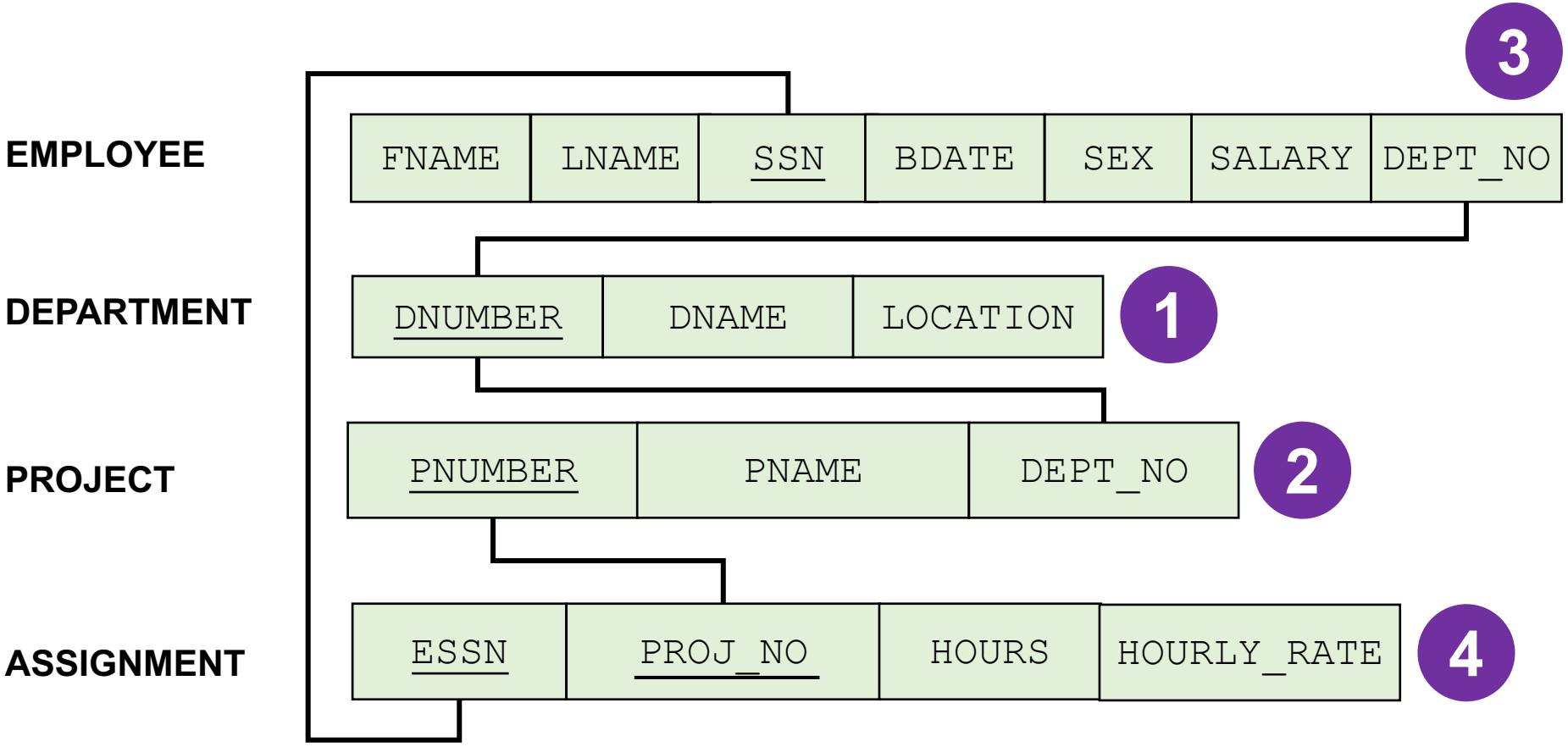
Exercise: CREATE

boxofficemovie(mid, title, studio, year, gross)

mid	title	studio	year	gross
1	Avengers: Endgame	BV	2019	2794.7
2	Avatar	Fox	2009	2789.7
3	Titanic	Par	1997	2187.5
4	Star Wars: The Force Awakens	BV	2015	2068.2

Write a DDL command to create `boxofficemovie` table

Sequence of CREATE



Example of SQL commands: tinycompany

```
CREATE DATABASE IF NOT EXISTS tinycompany;
USE tinycompany;

-- Department Table
CREATE TABLE department(
    dnumber             INT PRIMARY KEY,        -- dnumber is a primary key
    dname               VARCHAR(50) NOT NULL,
    location            VARCHAR(100),           -- location is nullable
);


-- Project Table
CREATE TABLE project(
    pnumber             INT PRIMARY KEY,        -- dnumber is a primary key
    pname               VARCHAR(50) NOT NULL,
    dept_no             INT NOT NULL,
    CONSTRAINT FK_DeptProj FOREIGN KEY (dept_no) REFERENCES department(dnumber)
);
```

SQL Command for `tinycompany` (cont.)

```
/* continue from the previous script */

-- Employee Table
CREATE TABLE employee(
    fname               VARCHAR(20) NOT NULL,
    lname               VARCHAR(20) NOT NULL,
    ssn                 CHAR(9) PRIMARY KEY,
    bdate               DATE NOT NULL,
    sex                 CHAR(1) NOT NULL,
    salary              DECIMAL(12,2),          -- salary is nullable
    dept_no             INT,                    -- dept_no is nullable
    CONSTRAINT FK_EmpDept FOREIGN KEY (dept_no) REFERENCES department(dnumber)
);

-- Assignment Table
CREATE TABLE assignment(
    essn                CHAR(9) NOT NULL,
    projno              INT NOT NULL,
    hours               DECIMAL(9,2),           -- hours is nullable
    hourlyrate          DECIMAL(9,2),           -- hourlyrate is nullable
    CONSTRAINT FK_AsmEmp FOREIGN KEY (essn)   REFERENCES employee(ssn),
    CONSTRAINT FK_AsmPrj FOREIGN KEY (projno) REFERENCES project(pnumber),
    PRIMARY KEY (essn, projno)                  -- Define a Composite Primary key
);
```

Modify an Existing Table

```
ALTER TABLE [table name]
ADD | MODIFY |DROP …
```

- Add a new column / constraint
- Modify a column / constraint
- Drop a column / constraint
- Modify an existing column / constraint
  - Add / Drop primary key
  - Add / Drop foreign key
  - Add / Drop `CHECK` constraint
  - Add / Drop `DEFAULT` constraint

Modify an Existing Table

- Add new columns

```
ALTER TABLE project
ADD signdate      DATE         NOT NULL,
ADD pstatus       CHAR(1)      NOT NULL;
```

- Modify columns

```
-- Change varchar to char (fixed length)
ALTER TABLE employee
MODIFY ssn CHAR(9) NOT NULL,
MODIFY sex CHAR(1) NOT NULL;
```

Modify an Existing Table

- Drop a column

```
ALTER TABLE project
DROP signdate;
```

Note: RDMBS will allow you to drop columns if they are not referenced by other tables / columns (enforced by referential integrity)

```
-- dnumber is PK and FK in the project table
ALTER TABLE department
DROP dnumber;
```

```
Error Code: 1829. Cannot drop column
'dnumber': needed in a foreign key
constraint 'FK_DeptProj' of table
'project'
```

Modify an Existing Table

- Modify a primary key

```
-- Remove a current primary key
ALTER TABLE employee
DROP PRIMARY KEY;
-- Create a new composite primary key
ALTER TABLE employee
ADD PRIMARY KEY (fname, lname);
```

- Modify a foreign key

```
ALTER TABLE employee
DROP CONSTRAINT FK_EmpDept; -- Remove FK
ALTER TABLE employee
ADD CONSTRAINT FK_EmpDept FOREIGN KEY
(dept_no) REFERENCES department(dnumber);
```

Modify an Existing Table

- Add a `CHECK` constraint (for data validation)

```
-- Check if sex has value 'M' or 'F'
ALTER TABLE employee
ADD CONSTRAINT CHK_Gender CHECK
(sex in ('M', 'F'));
```

- Add a `DEFAULT` constraint (for setting default value)

```
-- Set default location to 'Bangkok'
ALTER TABLE department
ALTER location SET DEFAULT 'Bangkok'
```

Example: ALTER `employee` Table

Employee	fname	Employee's first name	varchar(20)	Xxxxxx				
	lname	Employee's last name	varchar(20)	Xxxxxx				
	ssn	Social security number	char(9)	xxxxxxxxx			PK	
	bdate	Employee's birthday	date	yyyy-mm-dd				
	sex	Employee's gender	char(1)	X		M,F		
	salary	Salary	decimal(12,2)	1234567890.00	Y			
	dept_no	Department's number	int	x	Y		FK	dnumber [Department]

Add a new column

```
ALTER TABLE project
ADD signdate DATE NOT NULL;
```

Change the data type

```
ALTER TABLE Employee
MODIFY salary DECIMAL(14,2);
```

Add the **CHECK** constraint for data validation

```
ALTER TABLE Employee
ADD CONSTRAINT chk_sex CHECK (sex IN ('M', 'F')),
ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

Exercise: ALTER

boxofficemovie(**mid**, title, studio, year, gross)

studio(**sid**, sname, location)

If we have created a new studio table, and want to change an attribute in the **boxofficemovie** table. Write a DDL command to change the attribute **studio** to **sid**, which is a primary key of the **studio** table. The resulting schema is as followed:

boxofficemovie(**mid**, title, **sid**, year, gross)

Drop a Table / Schema

- Remove the existing table permanently from the database

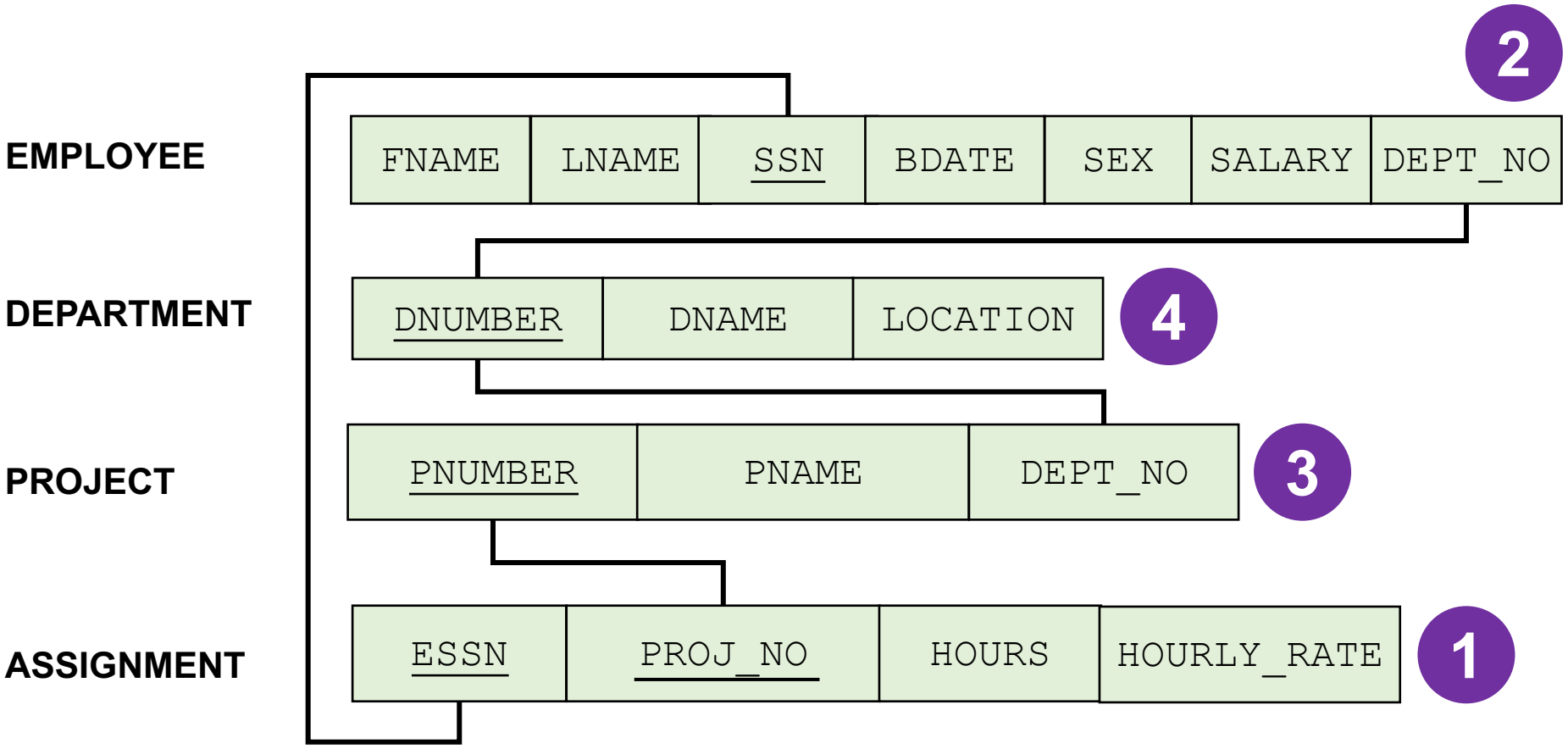
```
DROP TABLE [IF EXISTS] tablename;
```

- The table can be dropped only if it is not referenced by other tables.
  - RDBMS generates a foreign key integrity violation error message if the table is dropped

```
DROP TABLE IF EXISTS department;
```

```
Error Code: 3730. Cannot drop table 'department'
referenced by a foreign key constraint 'FK_DeptProj' on
table 'project'.      0.015 sec
```

Sequence of DROP



Exercise: DROP

boxofficemovie(**mid**, title, **sid**, year, gross)

studio(**sid**, sname, location)

Write a DDL command remove both `boxofficemovie` table **and** `studio` table.
Hint: Think about the drop sequence.

Mahidol University

SQL: Data Manipulation Language



DML: Data Manipulation Language

- Commands are used to manipulate data in a relational database.
  - `INSERT` - Used to create a record.
  - `UPDATE` - Used to change records.
  - `DELETE` - Used to delete records.

INSERT a new tuple

- Syntax :

```
INSERT INTO tablename(Column1, Column2,…,ColumnN)
VALUES (value1, value2, ... , valueN)
```

- Example:

Department Number: 7
Department name: IT
Location: 2Fl. 2B01

DEPARTMENT

DNUMBER	DNAME	LOCATION

```
INSERT INTO department (dnumber, dname, location)
VALUES (7, "IT", "2Fl. 2B01" );
```

or

You can ignore the list of column names, but your data must align with the columns

```
INSERT INTO department
VALUES (7, "IT", "2Fl. 2B01" );
```

INSERT a batch of tuples

- Insert a batch of data into a single table

DNUMBER	DNAME	LOCATION
1	Accounting	2A101 Fl.1
2	Human Resources	2A104 Fl.1
3	Research and Development	2B401 Fl.4
4	Information Technology	2A404 Fl.4
5	Public Relations	2B201 Fl.2
6	Administration	2B301 Fl.3
7	Academic Services	2B302 Fl.3

```
INSERT INTO department
VALUES (1, "Accounting", "2A101 Fl.1" ),
       (2, "Human Resources", "2A104 Fl.1" ),
       (3, "Research and Development", "2B401 Fl.4" ),
       (4, "Information Technology", "2A404 Fl.4" ),
       (5, "Public Relations", "2B201 Fl.2" ),
       (6, "Administration", "2B301 Fl.3" ),
       (7, "Academic Services", "2B302 Fl.3" );
```

UPDATE an existing tuple

- Syntax :

```
UPDATE      tablename
SET columnname=expression[, columnname=expression]
[WHERE      conditionlist ];
```

- Example: Update `dept_no` to be `2` in the `project` table

PNUMBER	PNAME	DEPT_NO
2	APTX4742	*NULL*

```
UPDATE project
SET dept_no=2
WHERE pnumber=2;
```

Normally, we use the primary key as a condition to **uniquely identify** the row to be updated

pnumber	pname	dept_no
1	APTX4869	3
2	APTX4742	NULL

→

pnumber	pname	dept_no
1	APTX4869	3
2	APTX4742	2

UPDATE an existing tuple (cont.)

- Example: Update Peter's `salary` and `dept_no`

FNAME	LNAME	SSN	BDATE	SEX	SALARY	DEPT_NO
Peter	Parker	110033445	1985-05-04	M	*NULL*	*NULL*

```
UPDATE employee
SET salary=1800.5, dept_no = 3
WHERE fname="Peter" AND lname="Parker";
```

Note: we can use this condition as long as we have only one `Peter Parker`.

```
UPDATE employee
SET salary=1800.5, dept_no = 3
WHERE ssn="110033445";  -- Use PK
```

fname	lname	ssn	bdate	sex	salary	dept_no
MaryJane	Watson	103849237	1983-08-19	F	3400.40	1
Peter	Parker	110033445	1985-05-04	M	NULL	NULL
Miles	Morales	230563445	1990-08-31	M	NULL	NULL

fname	lname	ssn	bdate	sex	salary	dept_no
MaryJane	Watson	103849237	1983-08-19	F	3400.40	1
Peter	Parker	110033445	1985-05-04	M	1800.50	3
Miles	Morales	230563445	1990-08-31	M	NULL	NULL

DELETE an existing tuple

- **Syntax :**

```
DELETE FROM tablename [WHERE conditionlist ];
```

- Example:

```
DELETE FROM assignment
WHERE essn="103849237" AND proj_no=1;
```

```
DELETE FROM department
WHERE location IS NULL;
```



What will happen if the `DELETE` command **without** the `WHERE` condition?

For example,

```
DELETE FROM department;
```

Mahidol University

Getting Started to MySQL Server and MySQL Workbench



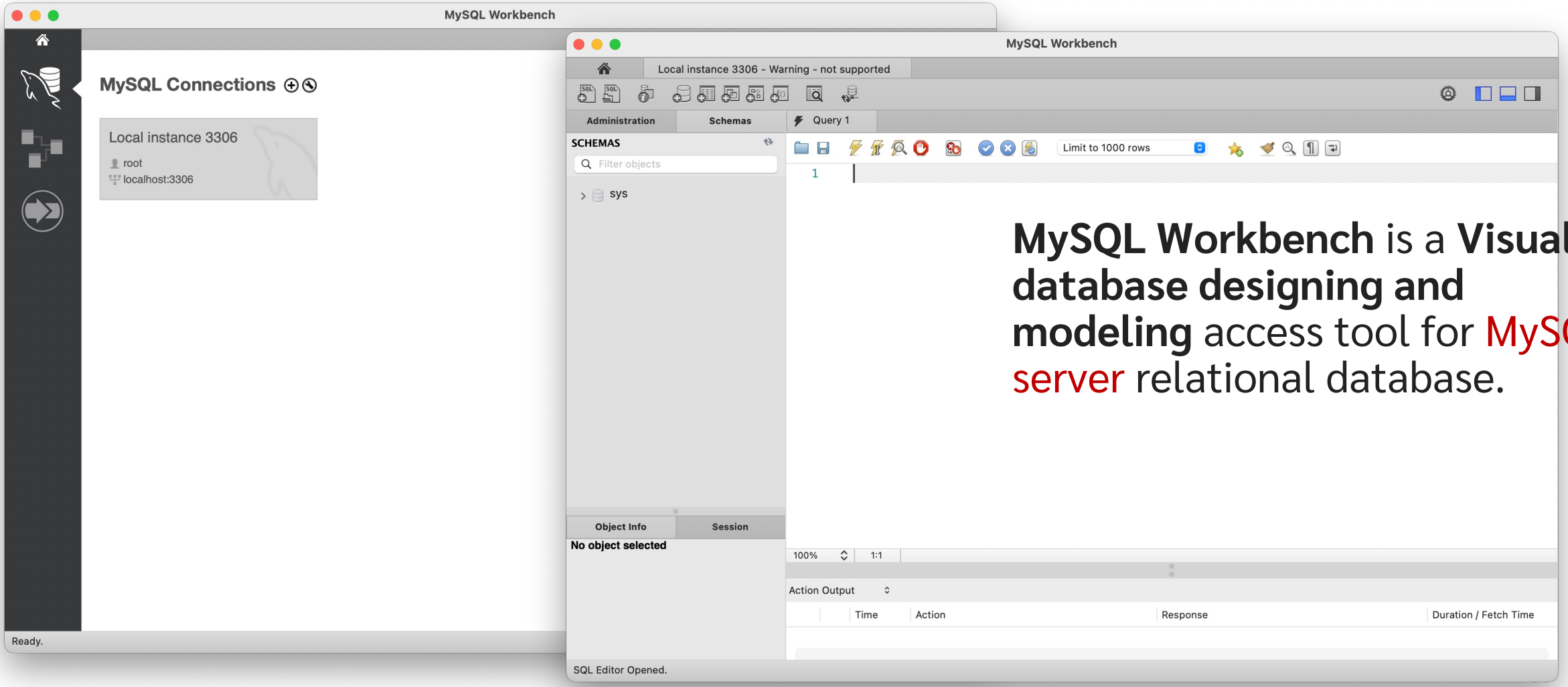
MySQL

- MySQL ("My S-Q-L") is an open-source relational database management system (RDBMS), developed by Oracle
- Relational database management system (RDBMS) is the software used to store, manage, query, and retrieve data stored in a relational database, providing an interface between users and applications and the database, as well as administrative functions for managing data storage, access, and performance.

https://www.mysql.com/



MySQL Server and Workbench



MySQL Workbench is a **Visual database designing and modeling** access tool for MySQL server relational database.

Things to install

Note: You need "Server" before Workbench

MySQL Community Downloads

- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository

- MySQL Community Server
- MySQL Cluster
- MySQL Router
- MySQL Shell
- MySQL Operator
- MySQL NDB Operator
- MySQL Workbench

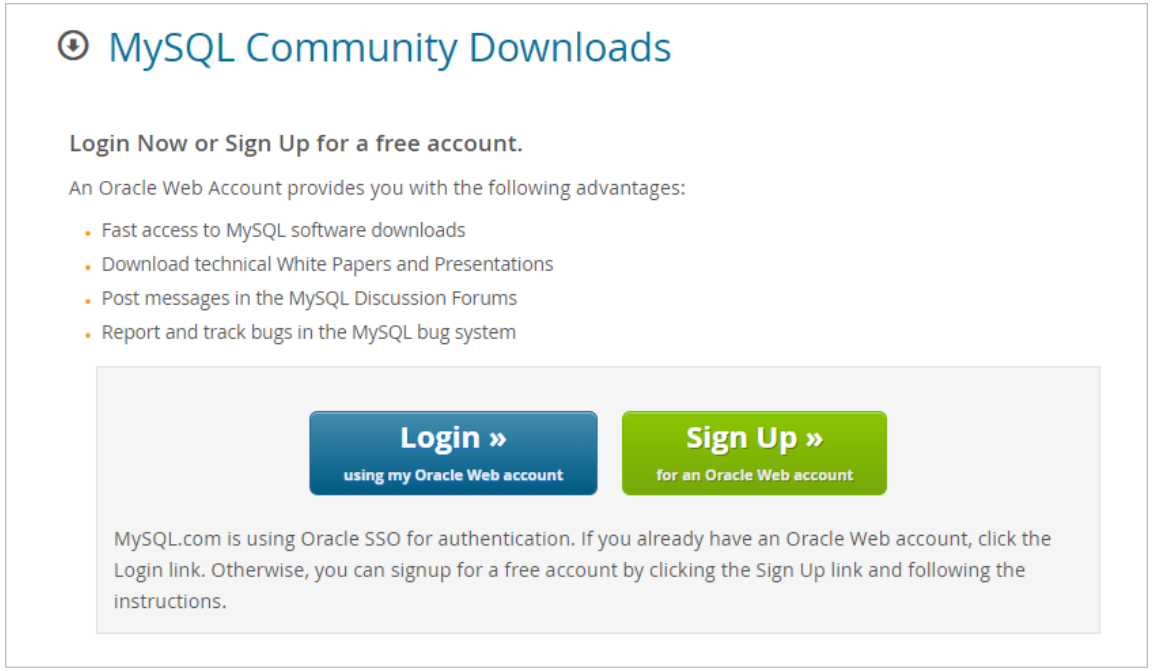
- MySQL Installer for Windows

- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP

- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

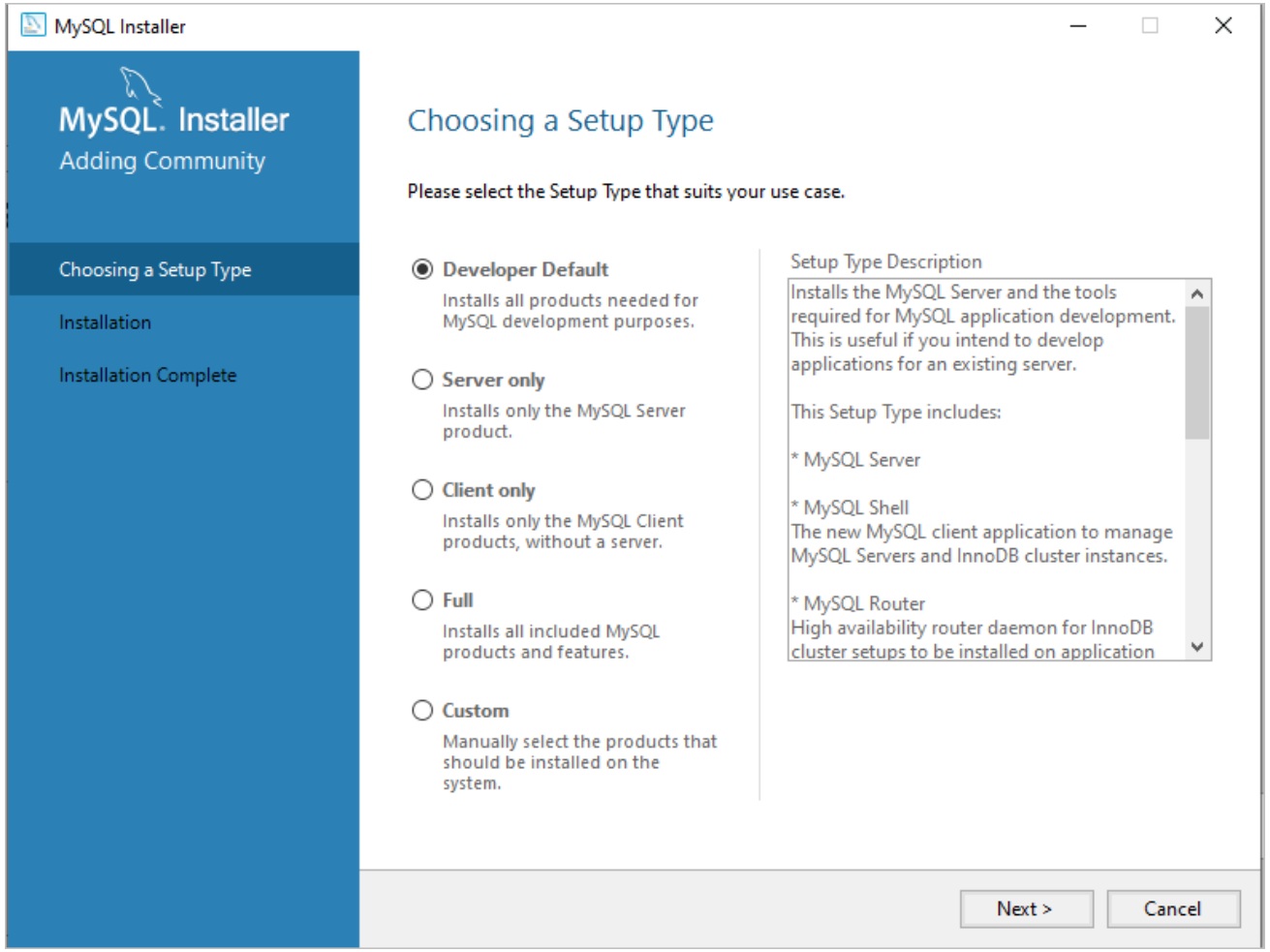
Register Oracle Account (Optional)

1. Register for an Oracle Account (using your MU email)
   - You can use your MU student emails
   - Use Faculty of ICT as your workplace contact
2. Verify your email
3. Download and install

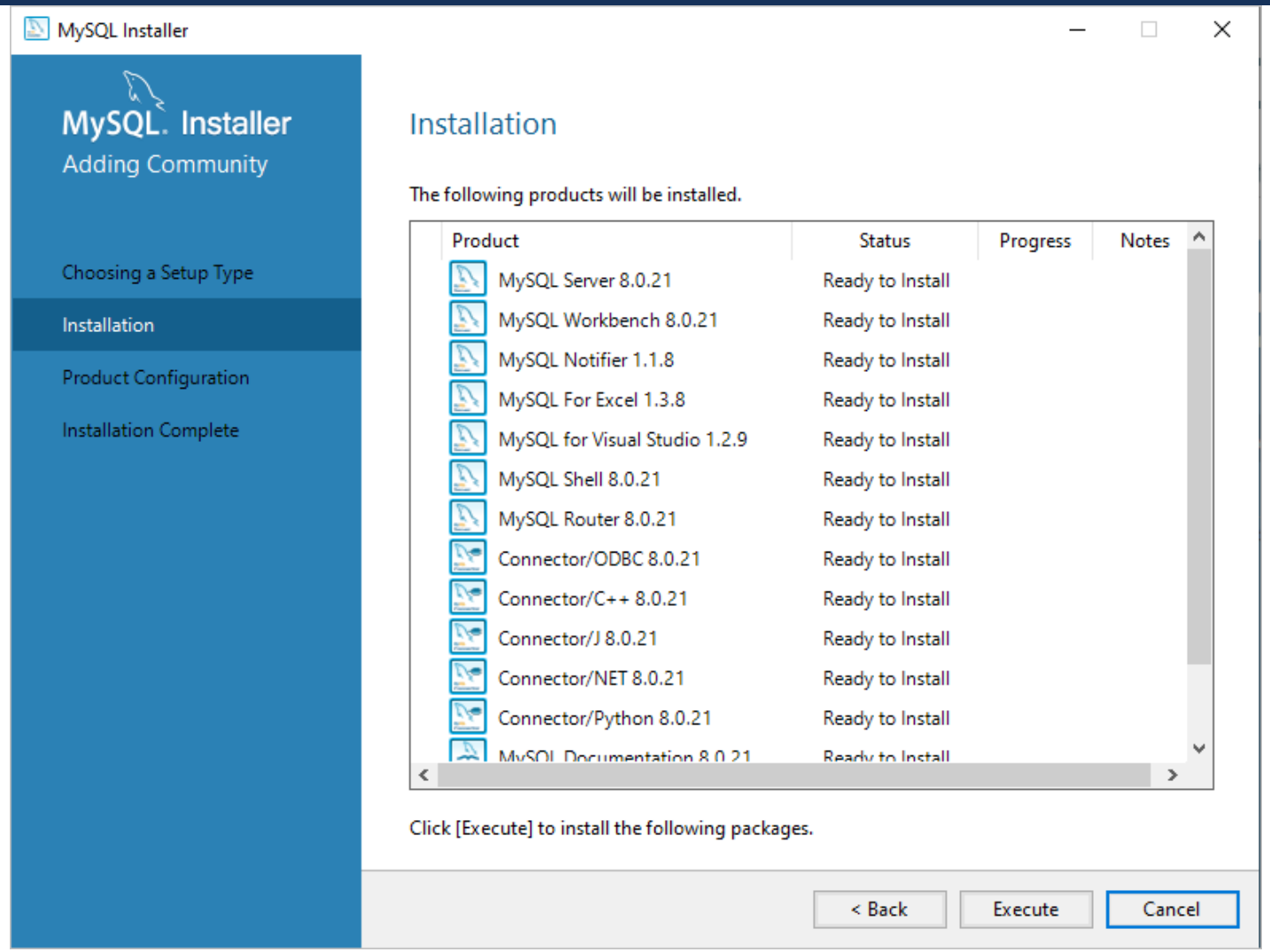


MySQL - Developer Default

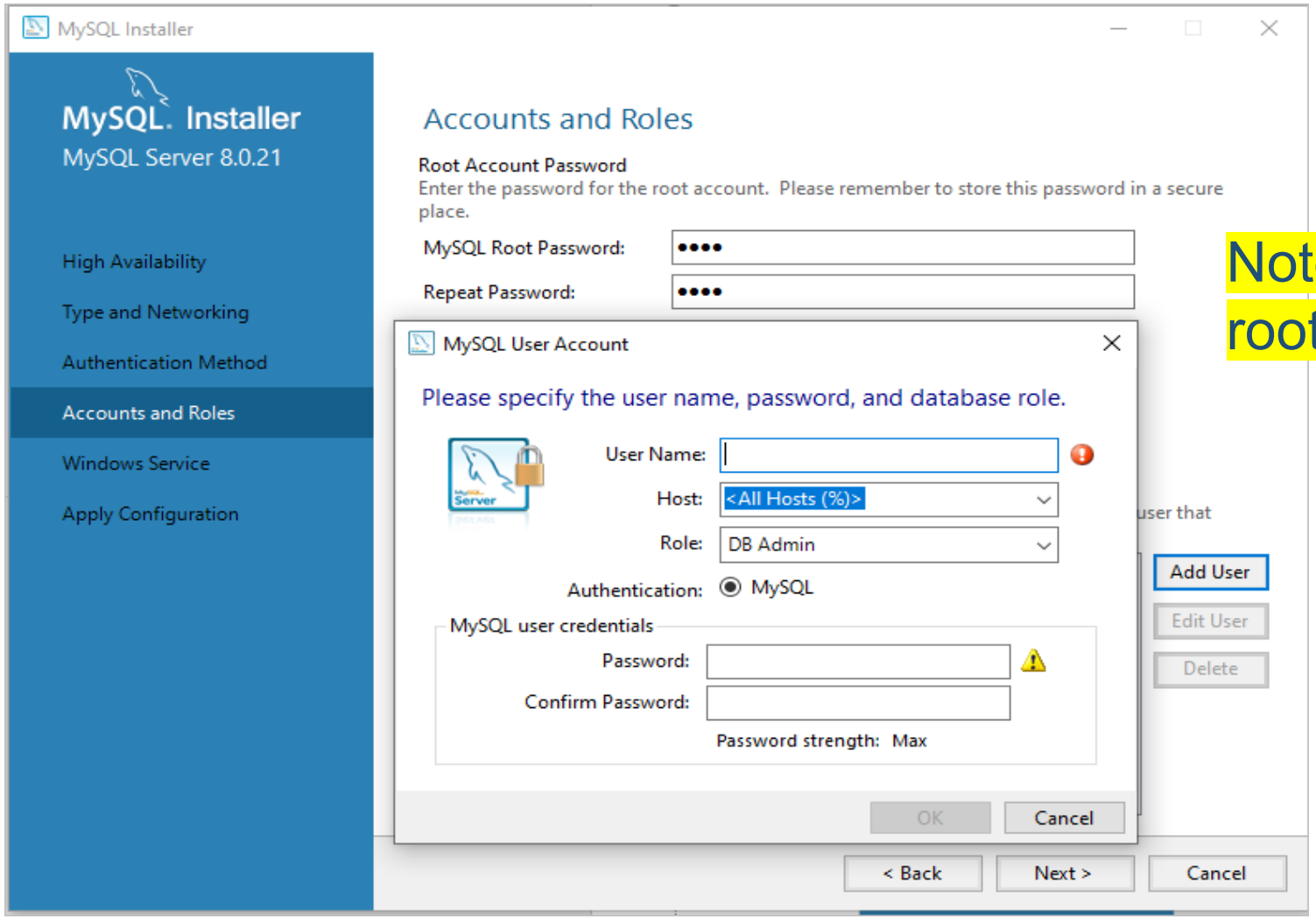
Note: Your computer may need to install extra packages required by the MySQL workbench



MySQL – Installation

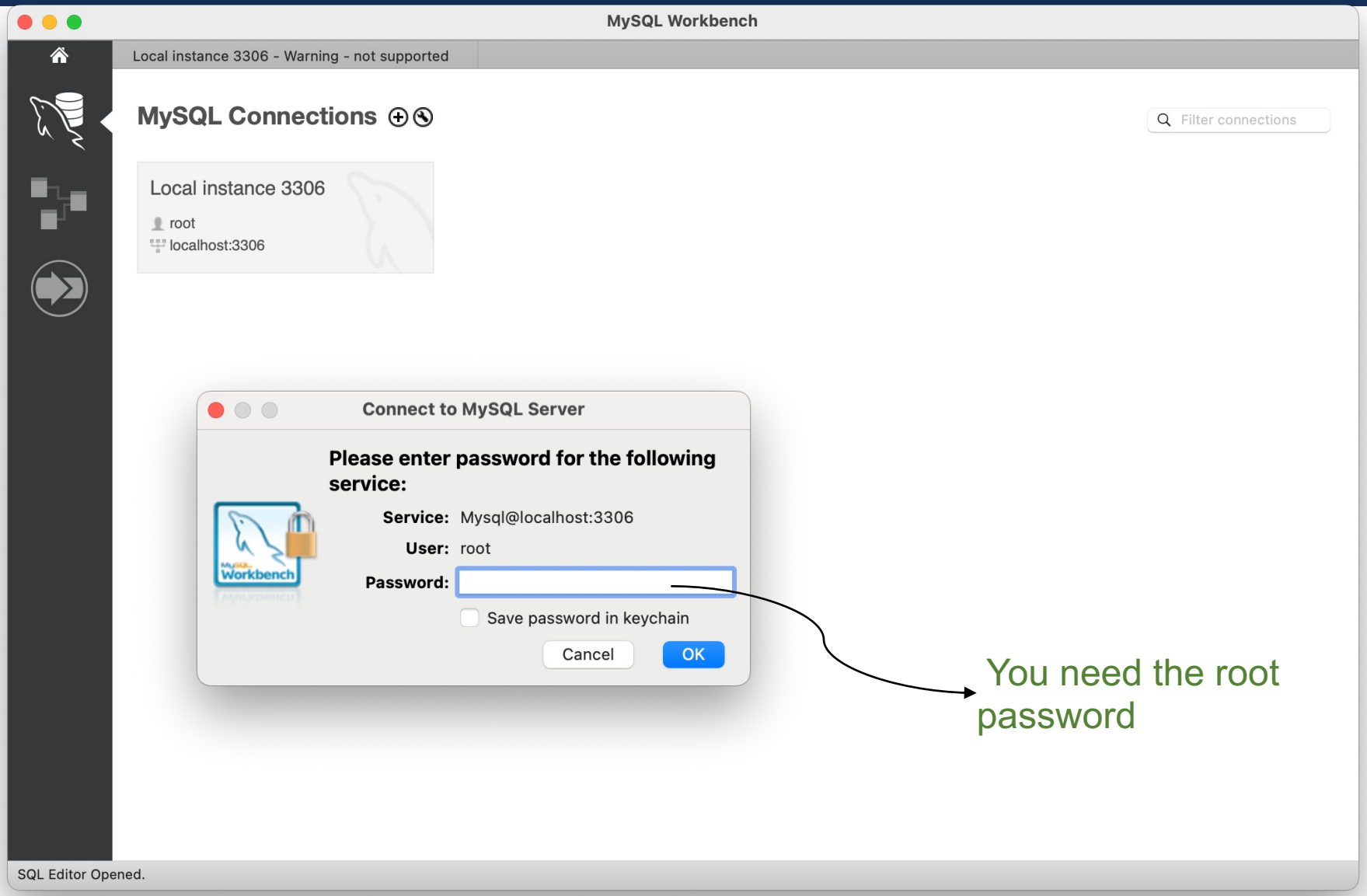


MySQL – Set Up Root User



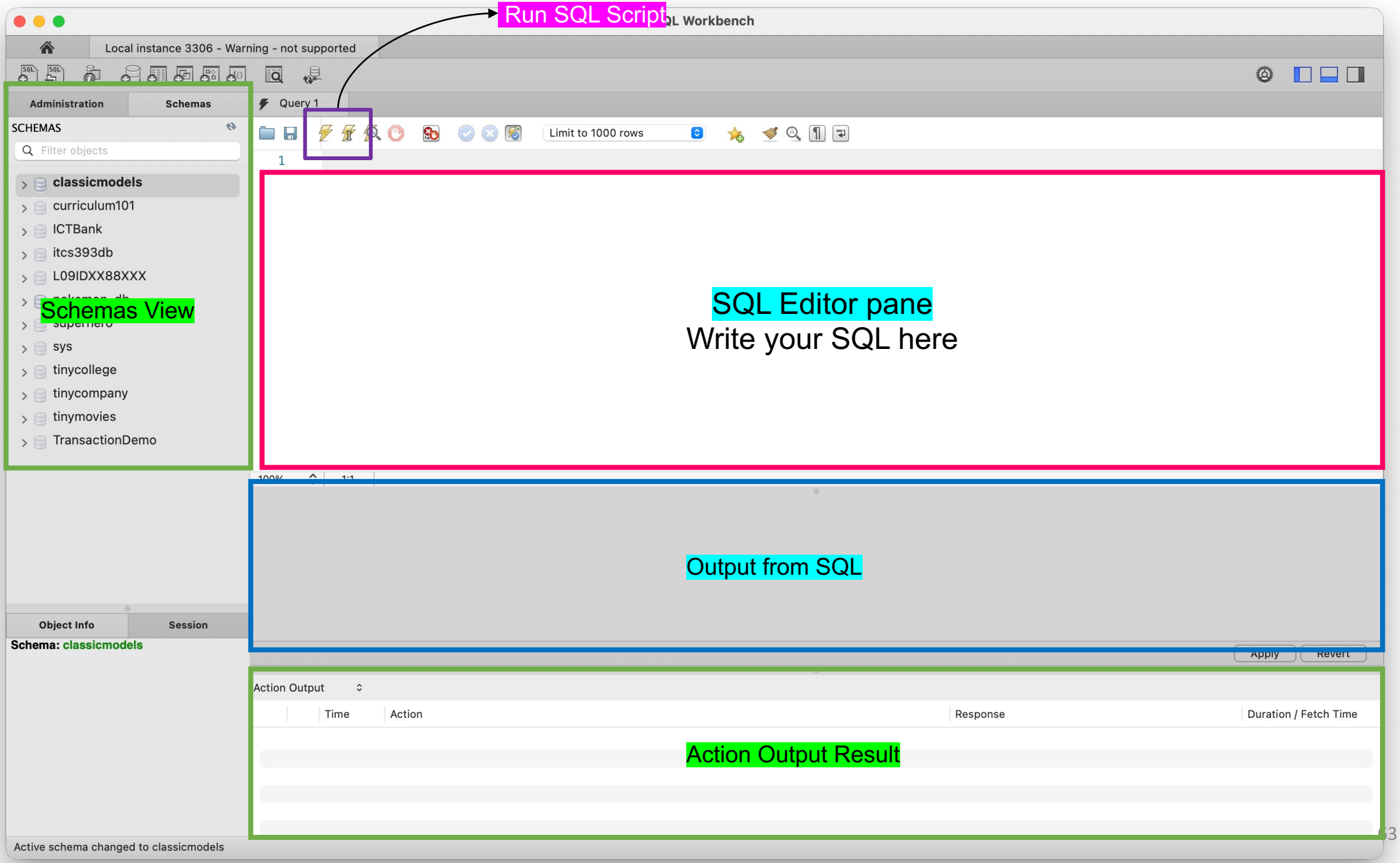
Note: Make sure you remember root and password

Start Your MySQL



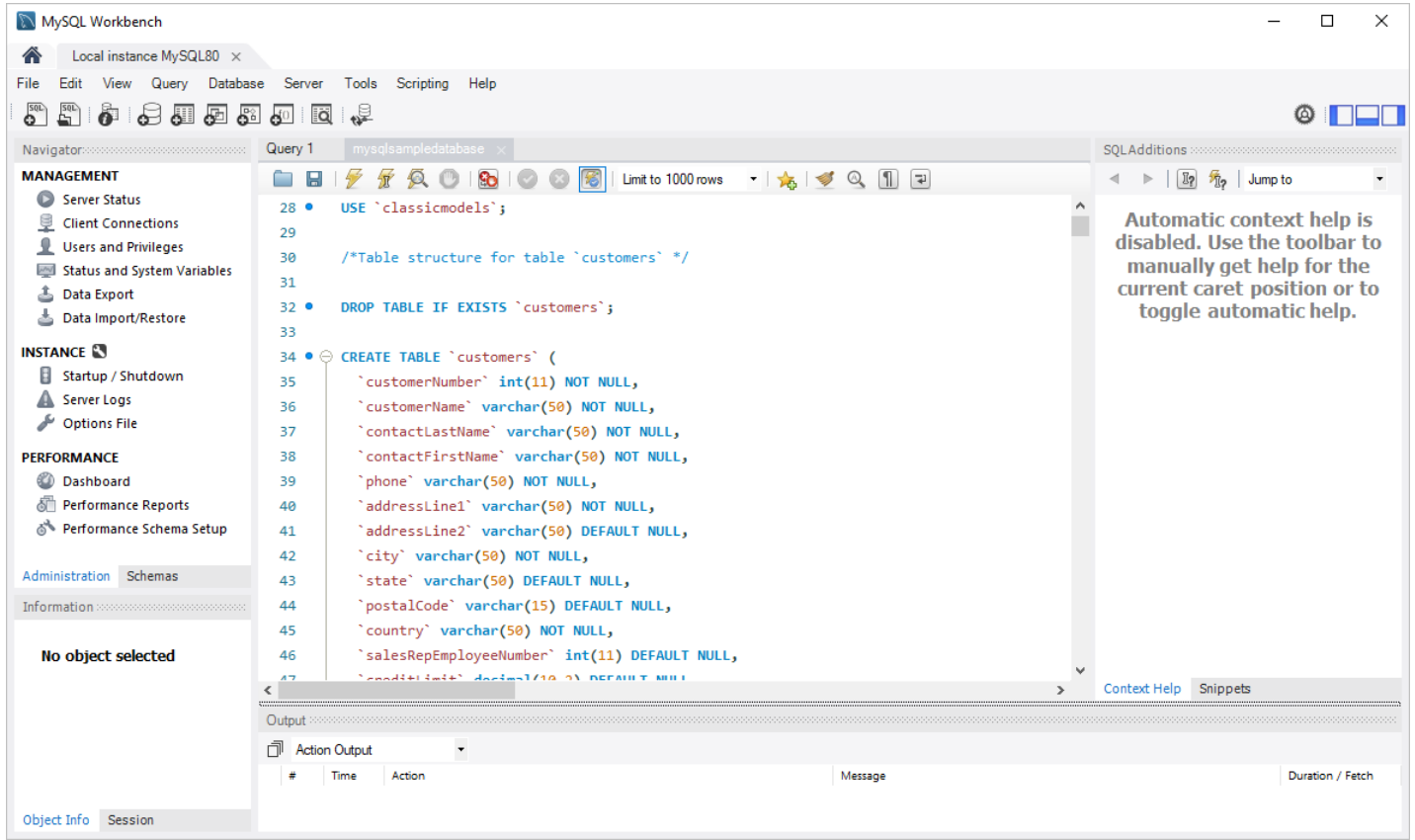
You need the root password

Note: If you open Workbench and there is no any “Local Instance” found, make sure MySQL server is installed correctly or you may need to manually start MySQL server [ref]


Run SQL Script
L Workbench
Local instance 3306 - Warning - not supported
Administration
Schemas
SCHEMAS
Filter objects
classicmodels
curriculum101
ICTBank
itcs393db
L09IDXX88XXX
sys
tinycollege
tinycompany
tinymovies
TransactionDemo
Schemas View
Query 1
Limit to 1000 rows
1
SQL Editor pane
Write your SQL here
Output from SQL
Object Info
Session
Schema: classicmodels
Apply
Revert
Action Output
Time
Action
Response
Duration / Fetch Time
Action Output Result
Active schema changed to classicmodels

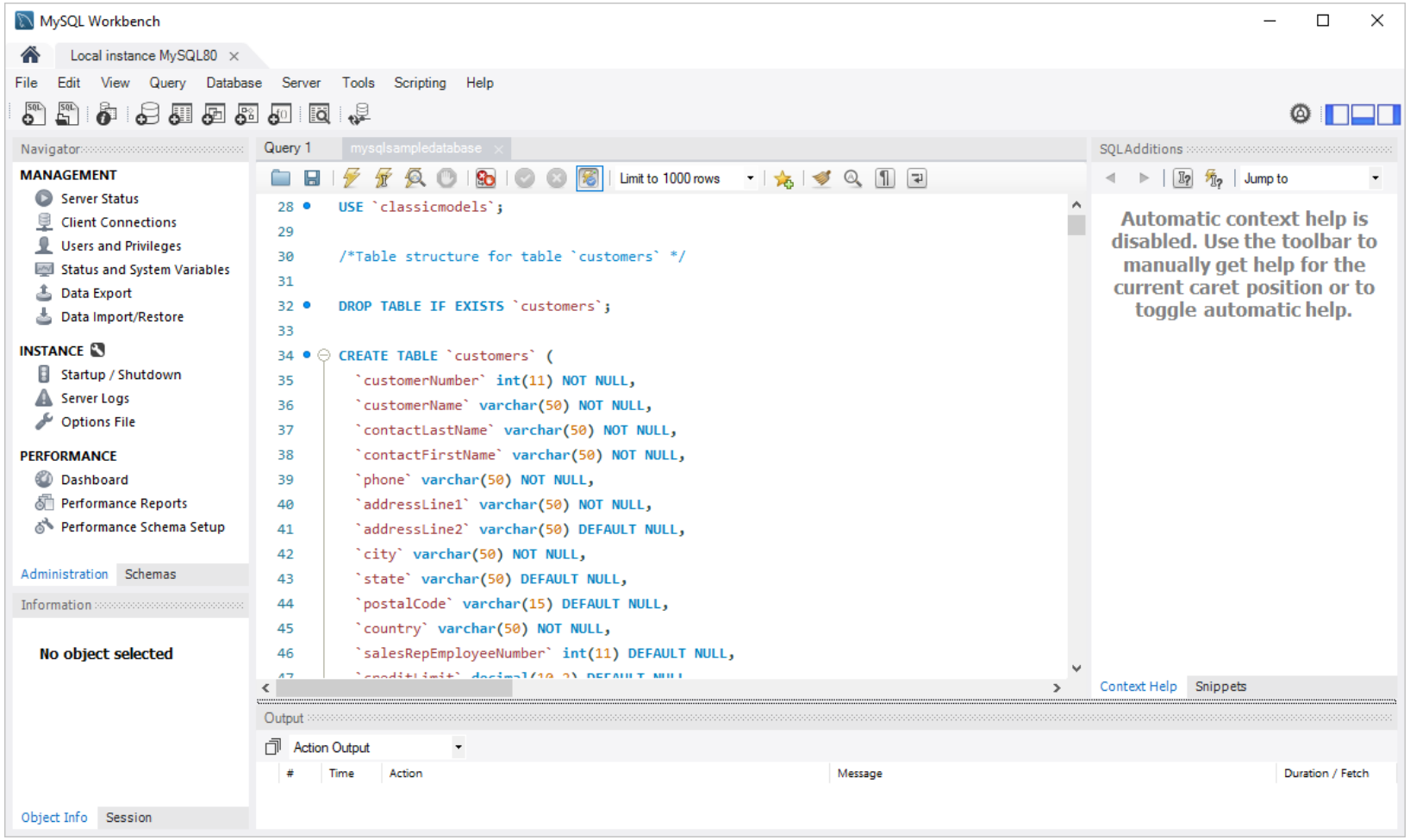
Create your First SQL Script

- Write the SQL Command in the Editor pane
- Go to “File” → “Save Script As…”
- Save your file as `.sql` and reopen if needed



Create a DB from the SQL file

Open SQL file > Execute the script file



Next week Workshop

- Bring your laptop for the group workshop;
  the MySQL server and the MySQL workbench must be installed
- Bring your report Phase I for refinement